MXEN3002

Mechatronic Automation Project

Laboratory Logbook, Semester 2, 2026

Bentley (Perth) Campus

Name: Harry Cassidy

Student ID:

Tutor / Lab session:

Date submitted:

Festo MPS stations (Distributing, Testing, Sorting) programmed in Omron Sysmac Studio, simulated in CIROS Mechatronics, with SCADA in Ignition. All control designs were developed, tested, and demonstrated by the student in the laboratory.

Contents

This logbook is organised **one section per milestone**, in claim order. Material shared by several milestones (I/O tables, station overviews) sits in a short preamble at the head of each station.

Marks summary

Session	Milestones claimed	Marks	Running
1 (Wk 2, 28 Jul 2026)	P.1, 1.1–1.8	14	14
2 (Wk 3, 4 Aug 2026)	1.9, 1.10, 1.11, S1.1, S1.2, 2.1, 2.2, 2.3	15	29
3 (Wk 4, 11 Aug 2026)	2.4–2.8, S2.1, S2.2	15	44
4 (Wk 5, 18 Aug 2026)	3.1–3.4, S3.1, S3.2, L.1	15	59
5 (Wk 6, 25 Aug 2026)	L.2, L.3	11	70
Total		**70**	**70**

Session dates are the planned schedule; each milestone's result box records the demonstrated outcome and tutor sign-off.

Project Management

Milestone P.1 Gantt chart & strategy

Strategy for maximum marks. The lab is worth 70 marks claimed in-lab, capped at 15 marks per 3-hour session, with milestones gated by prerequisites. Two facts drive the plan. First, the function blocks are **shared across stations**: `FB_Panel` on all three, `FB_Actuator_T1` on Distributing + Testing, `FB_Actuator_T2` on Distributing + Sorting, `FB_Actuator_T3` on Testing + Sorting. The winning move is to design the FBs generic and correct once, on Station 1, and reuse them unchanged. Second, each function block and station sequence was designed and desk-checked against its finite-state diagram before the lab (Appendix), so lab time went only to typing/importing, tuning the stall timers, building the Ignition screens, and demonstrating. Together these collapse the work into **five sessions to 70/70**, each filled to the 15-mark cap in prerequisite order.

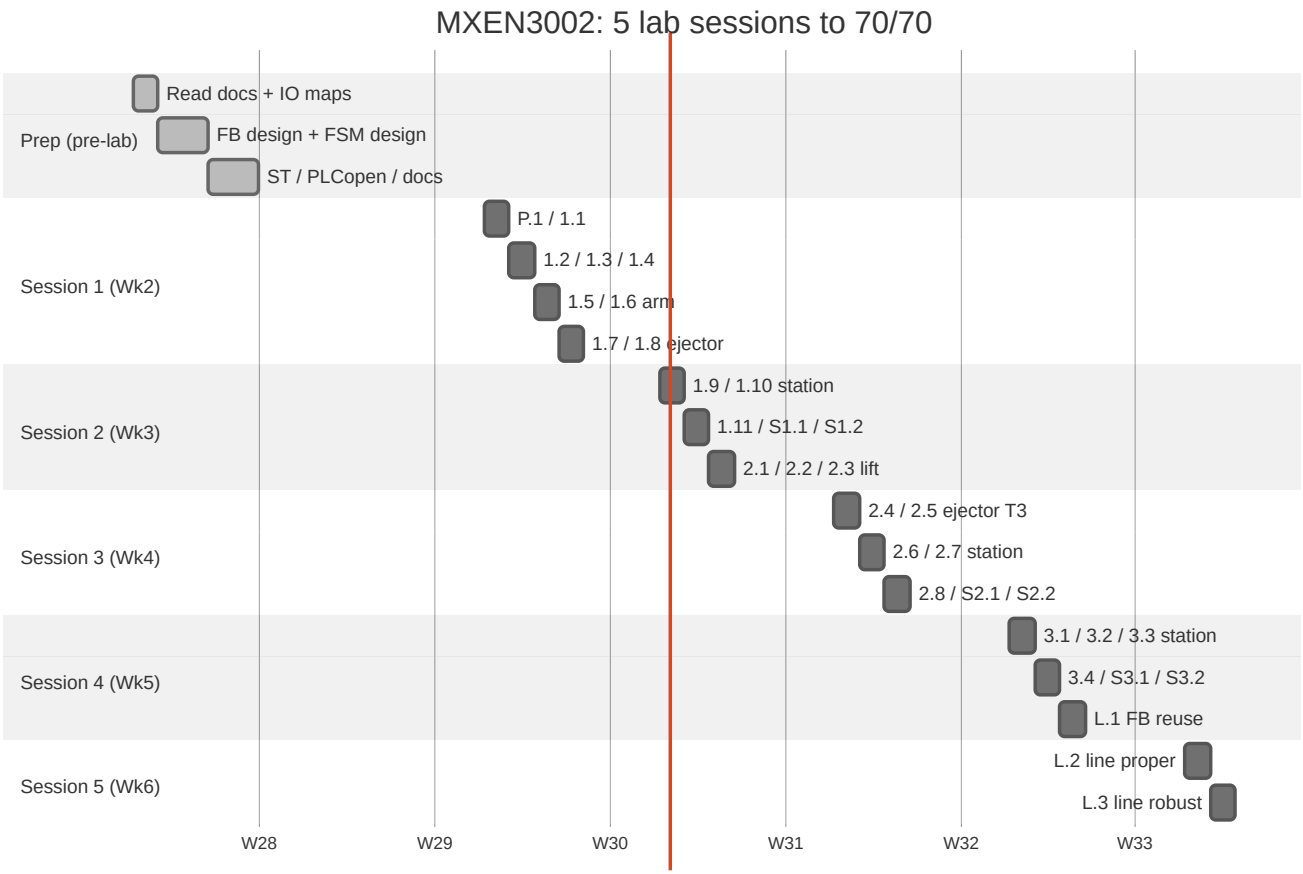

Figure 1. Project Gantt (five weekly sessions to 70/70)

> A colour-coded Excel version of this chart accompanies the project files (`00_plan/gantt.xlsx`). Dates above are the planned schedule.

The chart is revisited three times, once per station, as milestones 1.11, 2.8 and 3.4; each revision is recorded in its own section at the end of that station.

Distributing Station · Milestones 1.1–1.11

Station preamble: I/O table

Name	Addr	Function	Used by
Station_1B2	0.01	Ejection cylinder extended	T2 sensor_end
Station_1B1	0.02	Ejection cylinder retracted	T2 sensor_home
Station_2B1	0.03	Workpiece gripped (vacuum)	Main
Station_3S1	0.04	Arm at magazine (home)	T1 sensor_home
Station_3S2	0.05	Arm at downstream (end)	T1 sensor_end
Station_B4	0.06	Magazine empty	Main
IP_FI	0.07	Succeeding station free	Main
Panel_S1 / S2 / S4	0.08 / 0.09 / 0.11	Start / Stop / Reset	FB_Panel
Station_1Y1	1.00	Magazine eject	T2 valve_extend
Station_2Y1	1.01	Vacuum on	Main
Station_2Y2	1.02	Ejector pulse (blow-off)	Main
Station_3Y1	1.03	Arm to magazine	T1 valve_to_home
Station_3Y2	1.04	Arm to subsequent	T1 valve_to_end
Station_H1 / H2	1.08 / 1.09	Start / Reset light	FB_Panel
Panel_H3 / H4	1.10 / 1.11	Q1 / Q2	Main

Milestone 1.1 Tick-tock sampler

The supplied sampler program was downloaded, transferred to the station PLC and run to confirm the toolchain end to end: Sysmac Studio, the Ethernet link to the PLC, and the CIROS simulation all live at once.

Result, 1.1 tick-tock sampler. Downloaded `distributing_station_tick_tock.smc2`, connected to the PLC over Ethernet (134.7.44.xxx), transferred, set RUN, and observed the swivel arm cycling in CIROS with the Section0 rungs changing colour live. **PASS, 1/1.** Tutor: _____ Date: _____

Milestone 1.2 Station purpose

The Distributing station is the first station in the line. Its job is to **separate single workpieces from a stack magazine and feed them, one at a time, to the next station**. A double-acting **ejecting cylinder** pushes the lowest workpiece out of the gravity magazine into a pick-up position; a **swivel arm** with a vacuum suction cup then grips the workpiece, swivels it from the magazine to the downstream hand-over position, and releases it with an ejector (blow-off) pulse. Limit switches confirm each end position, a vacuum sensor confirms the grip, and a magazine-empty sensor stops the station when the stack is exhausted. Hand-over to the next station is coordinated by the `IP_FI` ("succeeding station free") signal.

Milestone 1.3 Operator interface

The station is operated from a panel of **three buttons** and **four lights**. Only one operational light (Start or Reset) is ever on, so the mode is readable at a glance.

Button	Addr	Function
Start	Panel_S1	From Pause → Run. Ignored in Reset/Error.
Stop	Panel_S2	Run → Pause; also Reset → Pause.
Reset	Panel_S4	Pause → Reset (home actuators); first step of error recovery.

Light	Addr	Meaning
Start light	Station_H1	ON = Run; OFF = Pause.
Reset light	Station_H2	ON = Reset (homing).
Q1	Panel_H3	Batch complete (5 workpieces delivered).
Q2	Panel_H4	Error state; all valves off, cleared by the reset sequence.

Milestone 1.4 Control panel : FB_Panel

The panel is a three-state machine shared by all three stations. Buttons request transitions; lights report the state. Invalid transitions have no edge and are therefore ignored.

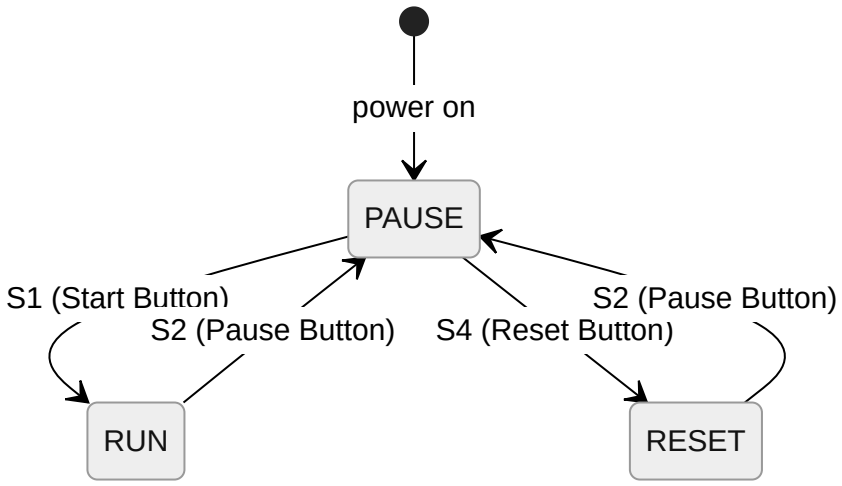

Figure 2. FB_Panel finite-state diagram (1.4)

Implemented in ladder as three latched state bits (set/reset by the transitions); the equivalent Structured Text is the canonical source:

```
FUNCTION_BLOCK FB_Panel
VAR_INPUT
    s1_start : BOOL; s2_stop : BOOL; s4_reset : BOOL;
END_VAR
VAR_OUTPUT
    run_mode : BOOL; reset_mode : BOOL; pause_mode : BOOL;
    start_light : BOOL; reset_light : BOOL;
END_VAR
VAR state : INT := 0; END_VAR   // 0 PAUSE 1 RUN 2 RESET

IF state = 0 THEN
    IF s1_start THEN state := 1; ELSIF s4_reset THEN state := 2; END_IF;
ELSIF state = 1 THEN
    IF s2_stop THEN state := 0; END_IF;
ELSIF state = 2 THEN
    IF s2_stop THEN state := 0; END_IF;
END_IF;

run_mode := (state=1); pause_mode := (state=0);
reset_mode := (state=2);
start_light := run_mode; reset_light := reset_mode;
END_FUNCTION_BLOCK
```

RESET exits only on Stop, which the invalid-transition steps require.

Result, 1.4. Power-on: all lights off (Pause). Start→Start light on (Run). Stop→off (Pause). Reset→Reset light on. Stop→off (Pause). Start pressed in Reset and Reset pressed in Run: no change, never two lights on. All six protocol steps observed. **PASS, 2/2.** Tutor: _____ Date: _____

Milestone 1.5 Swivel arm : FB_Actuator_T1 (proper)

The swivel arm is a **holdable double-solenoid** actuator: pause stops it mid-stroke and resume continues to the same target; reset drives it home.

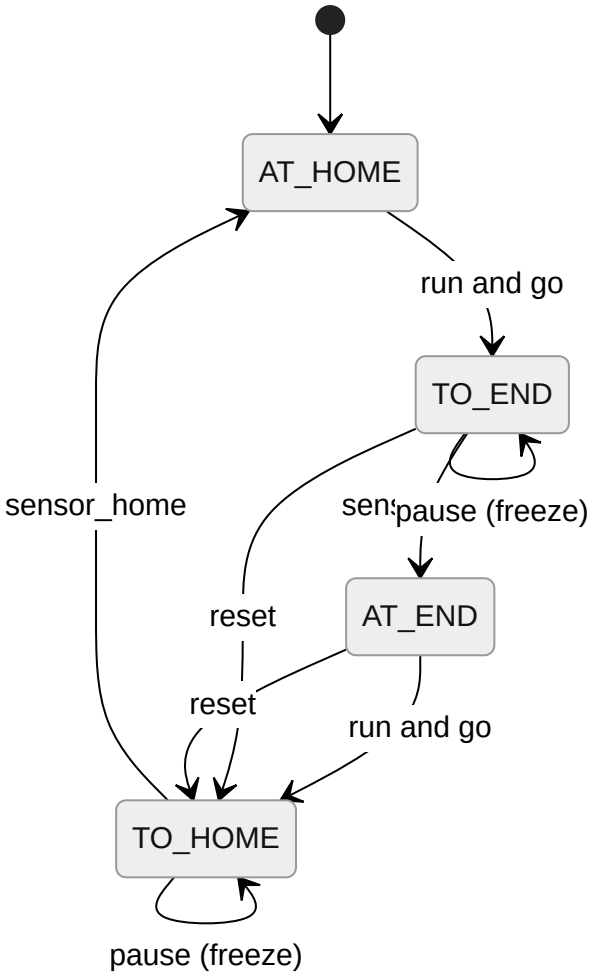

Figure 3. FB_Actuator_T1 proper (1.5)

One block covers both variants; the `robust` input arms the stall timer used by milestone 1.6 and is FALSE here.

```
FUNCTION_BLOCK FB_Actuator_T1   // holdable double-solenoid; arm and (reused) lift
VAR_INPUT
    run_mode : BOOL; pause_mode : BOOL; reset_mode : BOOL; error_mode : BOOL;
    sensor_home : BOOL; sensor_end : BOOL; go : BOOL;
    t_stall : TIME := T#2s; robust : BOOL := FALSE;
END_VAR
VAR_OUTPUT
    at_home : BOOL; to_end : BOOL; at_end : BOOL; to_home : BOOL;
    valve_to_end : BOOL; valve_to_home : BOOL; error : BOOL;
END_VAR
VAR state : INT := 0; stall : TON; moving : BOOL; END_VAR

CASE state OF
0:  IF error_mode THEN state := 4;
    ELSIF reset_mode THEN IF NOT sensor_home THEN state := 3; END_IF;
    ELSIF run_mode AND go THEN state := 1; END_IF;
1:  IF error_mode THEN state := 4;
    ELSIF reset_mode THEN state := 3;
    ELSIF pause_mode THEN ;
    ELSIF sensor_end THEN state := 2; END_IF;
2:  IF error_mode THEN state := 4;
    ELSIF reset_mode THEN state := 3;
    ELSIF run_mode AND go THEN state := 3; END_IF;
3:  IF error_mode THEN state := 4;
    ELSIF pause_mode AND NOT reset_mode THEN ;
    ELSIF sensor_home THEN state := 0; END_IF;
4:  IF reset_mode THEN IF sensor_home THEN state := 0; ELSE state := 3; END_IF; END_IF;
END_CASE;

valve_to_end  := (state = 1) AND NOT pause_mode;
valve_to_home := (state = 3) AND NOT pause_mode;
moving := valve_to_end OR valve_to_home;
stall(IN := robust AND moving, PT := t_stall);
IF stall.Q THEN state := 4; END_IF;
IF state = 4 THEN valve_to_end := FALSE; valve_to_home := FALSE; END_IF;

at_home := (state=0); to_end := (state=1); at_end := (state=2);
to_home := (state=3); error := (state=4);
END_FUNCTION_BLOCK
```

Result, 1.5 (proper). Reset with the arm in-between drove it home. In Run the arm cycled magazine ↔ downstream continuously. Stop mid-swivel froze it; Start resumed forward to the end without jerking home (both directions). Reset from a paused stroke returned it home. **PASS, 2/2.** Tutor: _____ Date: _____

Milestone 1.6 Swivel arm : FB_Actuator_T1 (robust)

The robust variant is the same block with `robust := TRUE` : a stall timer runs whenever a valve is driven, and forces the error state if the arm is held.

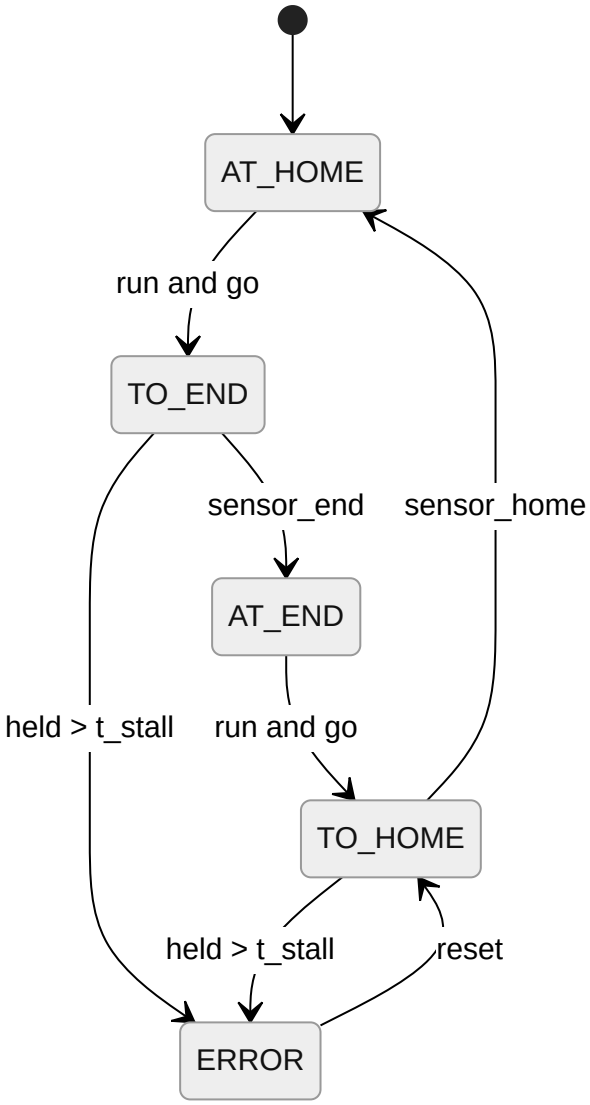

Figure 4. FB_Actuator_T1 robust (1.6)

Tuning (bench). Measured swivel travel ~0.8 s each way, so `t_stall` was left at `T#2s`. Healthy strokes never tripped it, and a manual hold reached Q2 at ~2 s as intended.

Result, 1.6 (robust). Holding the arm mid-swivel >2 s raised Q2 and stopped motion (home→end and end→home). Start in the error state did nothing; Reset then Start returned the arm to continuous motion. Q2 also lights when the arm is manually held. **PASS, 2/2.** Tutor: _____ Date: _____

Milestone 1.7 Ejection cylinder : FB_Actuator_T2 (proper)

The ejection cylinder is **stroke-completing**: pause takes effect only at the end of a stroke, so a Stop mid-travel completes to a limit switch and halts there.

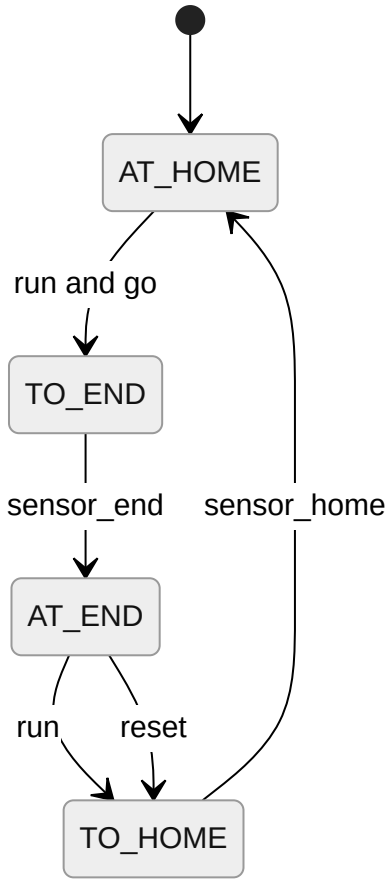

Figure 5. FB_Actuator_T2 proper (1.7)

```
FUNCTION_BLOCK FB_Actuator_T2   // stroke-completing, two sensors; ejector and (reused) branches
VAR_INPUT
    run_mode : BOOL; pause_mode : BOOL; reset_mode : BOOL; error_mode : BOOL;
    sensor_home : BOOL; sensor_end : BOOL; go : BOOL;
    t_stall : TIME := T#2s; robust : BOOL := FALSE;
END_VAR
VAR_OUTPUT
    at_home : BOOL; to_end : BOOL; at_end : BOOL; to_home : BOOL;
    valve_extend : BOOL; error : BOOL;
END_VAR
VAR state : INT := 0; stall : TON; moving : BOOL; END_VAR

CASE state OF
0:  IF error_mode THEN state := 4;
    ELSIF reset_mode THEN IF NOT sensor_home THEN state := 3; END_IF;
    ELSIF run_mode AND go THEN state := 1; END_IF;
1:  IF error_mode THEN state := 4; ELSIF sensor_end THEN state := 2; END_IF;
2:  IF error_mode THEN state := 4;
    ELSIF reset_mode THEN state := 3;
    ELSIF run_mode THEN state := 3; END_IF;
3:  IF error_mode THEN state := 4; ELSIF sensor_home THEN state := 0; END_IF;
4:  IF reset_mode THEN IF sensor_home THEN state := 0; ELSE state := 3; END_IF; END_IF;
END_CASE;

valve_extend := (state = 1) OR (state = 2);
IF state = 4 THEN valve_extend := FALSE; END_IF;
moving := (state = 1) OR (state = 3);
stall(IN := robust AND moving, PT := t_stall);
IF stall.Q THEN state := 4; valve_extend := FALSE; END_IF;

at_home := (state=0); to_end := (state=1); at_end := (state=2);
to_home := (state=3); error := (state=4);
END_FUNCTION_BLOCK
```

Result, 1.7 (proper). Reset retracted the cylinder fully home. In Run it extended and retracted continuously. A Stop mid-travel did not halt mid-stroke. The cylinder completed to the limit and halted in Pause (both directions); Start began the next stroke. **PASS, 2/2.** Tutor: _____ Date: _____

Milestone 1.8 Ejection cylinder : FB_Actuator_T2 (robust)

Same block with `robust := TRUE` ; the stall timer runs on both the extend and retract legs.

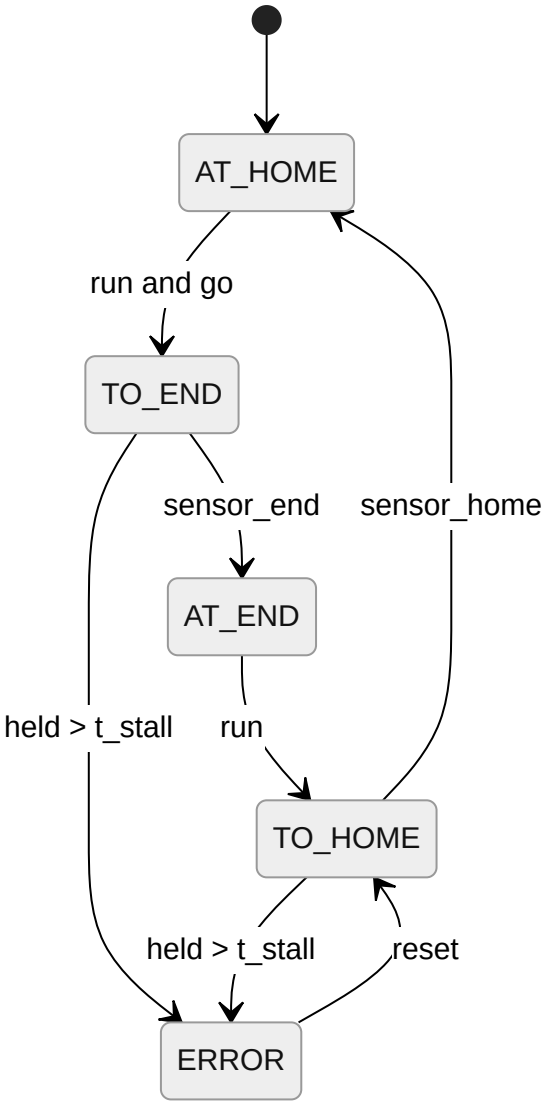

Figure 6. FB_Actuator_T2 robust (1.8)

Result, 1.8 (robust). Holding the cylinder (and, separately, switching off the air) mid-travel >2 s raised Q2, turned all valves off, and stopped motion (both directions). Start in error: no change; Reset then Start resumed continuous motion. **PASS, 2/2.** Tutor: _____ Date: _____

Milestone 1.9 Full station : Main (proper)

The fourth block (Main) owns the 7-step cycle, the vacuum and ejector-pulse valves, the unused I/O (`Station_B4`, `IP_FI`), the Q1/Q2 lights, and the 5-piece batch counter, all the I/O the actuator FBs must not own, including the constraint that `Station_2Y1/2Y2` stay out of `FB_Actuator_T1`.

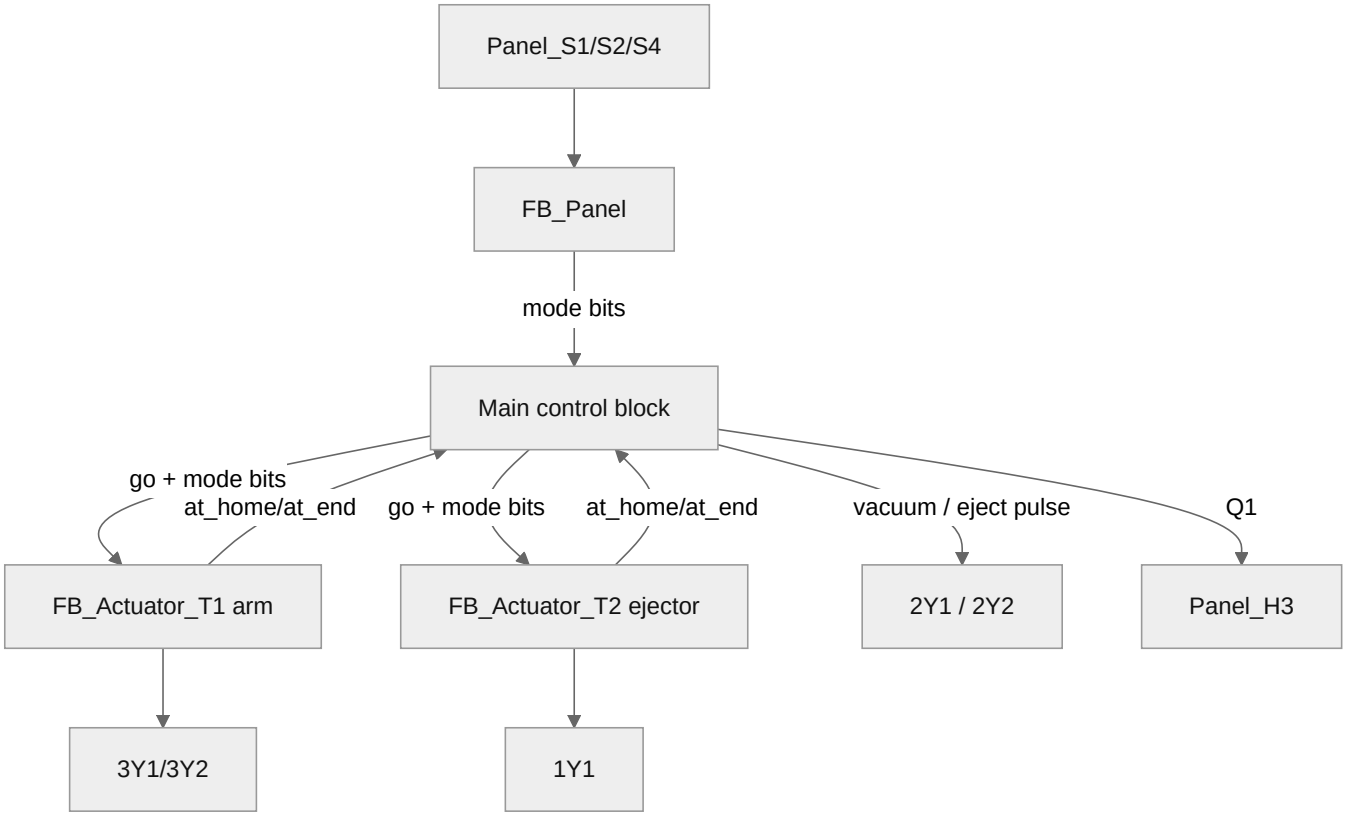

Figure 7. Distributing block diagram (4 blocks, 1.9)

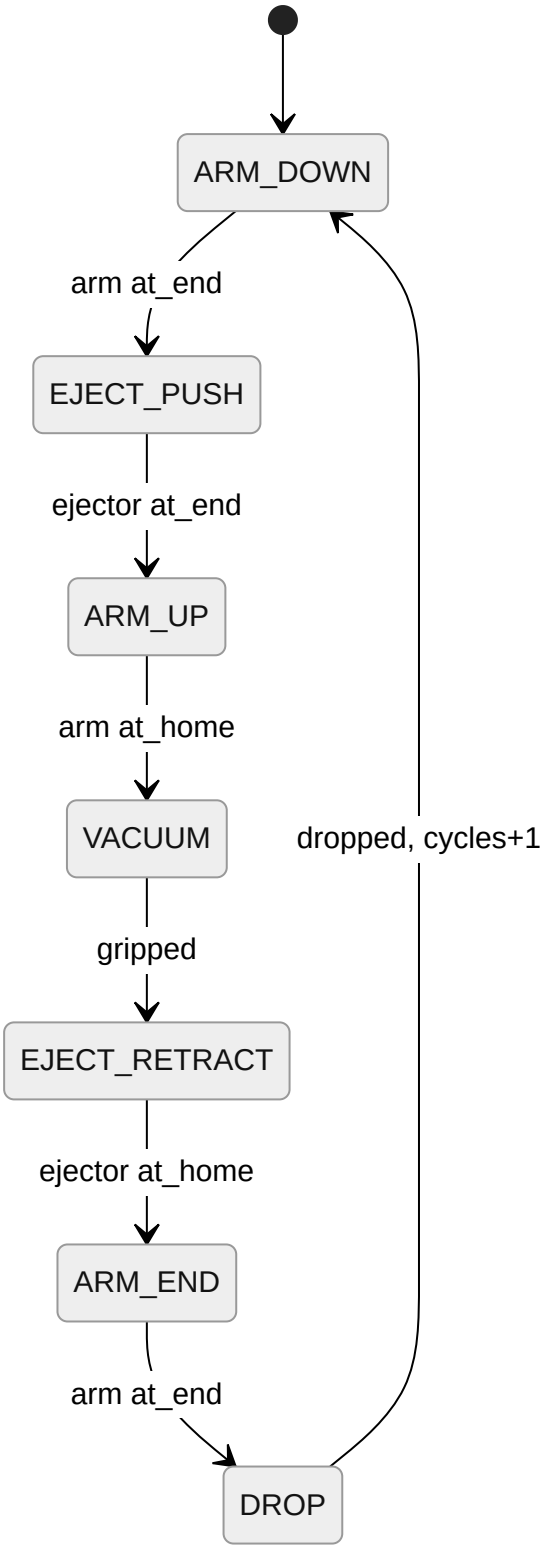

Figure 8. Distributing 7-step cycle (1.9)

```
PROGRAM Main_Distributing
VAR
    Panel : FB_Panel; Arm : FB_Actuator_T1; Ejector : FB_Actuator_T2;
    si : INT := 0; cycles : INT := 0; grip, drop : INT := 0;
    go_arm, go_ej, vacuum, eject_pulse, fault, reset_done : BOOL;
    ROBUST : BOOL := TRUE;   // 1.10 = TRUE, 1.9 = FALSE
    MAG : INT := 5;
END_VAR
reset_done := Arm.at_home AND Ejector.at_home;
Panel(s1_start := Panel_S1, s2_stop := Panel_S2, s4_reset := Panel_S4,
      error_in := fault, reset_done := reset_done);
IF Panel.reset_mode OR Panel.error_mode THEN
    si := 0; grip := 0; drop := 0; vacuum := FALSE; eject_pulse := FALSE;
    IF Panel.reset_mode THEN cycles := 0; END_IF;
END_IF;
go_arm := FALSE; go_ej := FALSE; eject_pulse := FALSE;
IF Panel.run_mode AND cycles < MAG THEN
    CASE si OF
    0:  go_arm := TRUE;  IF Arm.at_end THEN si := 1; END_IF;
    1:  go_ej  := TRUE;  IF Ejector.at_end THEN si := 2; END_IF;
    2:  go_arm := TRUE;  IF Arm.at_home THEN si := 3; END_IF;
    3:  vacuum := TRUE;  grip := grip+1; IF grip >= 1 THEN grip := 0; si := 4; END_IF;
    4:  go_ej := FALSE;  IF Ejector.at_home THEN si := 5; END_IF;
    5:  go_arm := TRUE;  IF Arm.at_end THEN si := 6; END_IF;
    6:  eject_pulse := TRUE; vacuum := FALSE; drop := drop+1;
        IF drop >= 1 THEN drop := 0; cycles := cycles+1; si := 0; END_IF;
    END_CASE;
END_IF;
Arm(run_mode := Panel.run_mode, pause_mode := Panel.pause_mode,
    reset_mode := Panel.reset_mode, error_mode := Panel.error_mode,
    sensor_home := Station_3S1, sensor_end := Station_3S2, go := go_arm,
    t_stall := T#2s, robust := ROBUST);
Ejector(run_mode := Panel.run_mode, pause_mode := Panel.pause_mode,
    reset_mode := Panel.reset_mode, error_mode := Panel.error_mode,
    sensor_home := Station_1B1, sensor_end := Station_1B2, go := go_ej,
    t_stall := T#2s, robust := ROBUST);
fault := ROBUST AND ( Arm.error OR Ejector.error
    OR (Panel.run_mode AND vacuum AND NOT Station_2B1)
    OR (Panel.run_mode AND Arm.at_home AND Station_B4 AND si <> 0) );
Station_3Y2 := Arm.valve_to_end;   Station_3Y1 := Arm.valve_to_home;
Station_1Y1 := Ejector.valve_extend;
Station_2Y1 := vacuum      AND NOT Panel.error_mode;
Station_2Y2 := eject_pulse AND NOT Panel.error_mode;
Station_H1  := Panel.start_light;  Station_H2 := Panel.reset_light;
Panel_H3    := (cycles >= MAG);    Panel_H4  := Panel.error_mode;
END_PROGRAM
```

Result, 1.9 (proper). Reset + 5 workpieces loaded: arm homed, ejector retracted. In Run the 7-step loop ran five times (arm downstream, ejector push, arm upstream, vacuum, ejector retract, arm to end, drop). After the 5th piece Q1 lit, both actuators returned home, vacuum off, station stayed in Run. Pause while the arm moved to delivery froze the arm while the ejector finished its stroke; Start resumed where it left off (both delivery and pickup). **PASS, 3/3.** Tutor: _____ Date: _____

Milestone 1.10 Full station : Main (robust)

Same program with `ROBUST := TRUE`. On top of the two actuators' stall timers, Main raises a fault when the vacuum reports no workpiece while gripping, and when the arm reaches an empty magazine mid-cycle.

Result, 1.10 (robust). Holding the arm, then the ejector, >2 s each raised Q2. Pulling a piece off the vacuum surface while active raised Q2; the arm reaching the magazine with no piece available raised Q2. In every error state all valves were off; recovery via Reset then Start. **PASS, 3/3.** Tutor: _____ Date: _____

Milestone 1.11 Gantt revision (after Station 1)

Sessions 1–2 landed 1.1–1.10 as planned. The only slippage was ~20 min tuning the swivel-arm stall timer on real hardware; absorbed within the session. Confirmed `FB_Panel` and `FB_Actuator_T1/T2` were written generic enough to reuse on later stations, the key objective of the front-loaded design.

SCADA : Distributing Station · Milestones S1.1, S1.2

Milestone S1.1 SCADA purpose

The Ignition SCADA interface is a supervisory window onto the Distributing station: on-screen **Start / Stop / Reset** drive the same `FB_Panel` state machine as the physical panel; the **start / reset / Q1** lamps mirror the panel lights; and an animated schematic shows the live positions of the swivel arm (in-magazine / in-between / in-downstream) and ejecting cylinder (home / in-between / end), with the workpiece moving through the station in real time.

Milestone S1.2 SCADA build

Tags were created as OPC tags against the PLC variables already entered in the Sysmac I/O map, so the names line up 1:1:

Ignition tag	PLC var	Dir	Purpose
Dist/Start_PB, Stop_PB, Reset_PB	Panel_S1/S2/S4	write	momentary buttons
Dist/StartLight, ResetLight, Q1, Q2	Station_H1/H2, Panel_H3/H4	read	lamps
Dist/Arm_Magazine, Arm_Downstream	Station_3S1 / 3S2	read	arm limits
Dist/Ej_Retracted, Ej_Extended	Station_1B1 / 1B2	read	cylinder limits
Dist/Vacuum	Station_2B1	read	workpiece gripped

The "in-between" state (no sensor) is derived per actuator by an expression tag, e.g. `if(Arm_Magazine, 0, if(Arm_Downstream, 2, 1))`. The arm graphic's rotation binds to that value (0° magazine, 90° downstream, 45° in-between); the cylinder bar's position binds to the ejector state; the workpiece rectangle follows whichever actuator is carrying it; everything turns red while Q2 is on.

Result, S1.1 / S1.2. With the 1.9 program running, the HMI Start button drove the station into the 7-step cycle; the arm and cylinder animated through their three positions and the workpiece moved magazine→ downstream, repeating five times with Q1 lighting after the fifth. Stop froze the arm on screen; Reset animated the actuators home. All required elements present (3 buttons, 3 lamps, both actuators' positions, workpiece positions, animation). **PASS, S1.1 1/1, S1.2 4/4.** Tutor: _____ Date: _____

Testing Station · Milestones 2.1–2.8

Station preamble: I/O table

Name	Addr	Function	Used by
Part_AV	0.00	Workpiece available	Main
Station_B2	0.01	Workpiece not black	Main (black := NOT B2)
Station_B4	0.02	Safety through-beam (TRUE=clear)	Main
Station_B5	0.03	Height correct (tall)	Main
Station_1B1 / 1B2	0.04 / 0.05	Lift up (end) / down (home)	T1
Station_2B1	0.06	Ejector retracted	T3 sensor_home
Station_1Y1 / 1Y2	1.00 / 1.01	Lift down / up	T1 valves
Station_2Y1	1.02	Advance ejecting cylinder	T3 valve_extend
Station_3Y1	1.03	Air cushion (blower)	Main (~1 s)
Station_H1/H2/H3/H4	1.08–1.11	Start / Reset / Q1 / Q2	Panel + Main

Lift convention: home = down (`1B2` / `1Y1`), end = up (`1B1` / `1Y2`).

Milestone 2.1 Station purpose

The Testing station is the second station: it **inspects each workpiece and sorts it by material/colour and height**. A double-acting **lift** raises the piece to a height sensor; the combination of an optical "not-black" sensor and the height reading classifies each piece as black, white (short) or red/metal (tall). An **ejecting cylinder** then pushes the piece onto the upper or lower slide, with a brief **air-cushion blower** assisting the upper-slide transfer. A safety through-beam guards the moving lift. The station **reuses** the shared `FB_Panel` and, for the lift, `FB_Actuator_T1` unchanged.

Milestone 2.2 Lift : FB_Actuator_T1 reused (proper)

The lift is the **same** `FB_Actuator_T1` block from the Distributing arm, wired to the lift's I/O. Its FSM (Figures 3–4) and behaviour are identical; only the call-site sensors and valves differ.

> **Result, 2.2 (proper).** Reset drove the lift to its bottom home. In Run it travelled up↔down; Stop mid-travel froze it and Start resumed (up and down). Demonstrated with the identical FB used on Station 1. **PASS, 1/1.** Tutor: _____ Date: _____

Milestone 2.3 Lift : FB_Actuator_T1 reused (robust)

Same block with `robust := TRUE` , so the stall timer applies to the lift exactly as it does to the swivel arm.

> **Result, 2.3 (robust).** Switching off the compressed air mid-travel for >2 s raised Q2 and stopped the lift; recovery via Reset then Start. **PASS, 1/1.** Tutor: _____ Date: _____

Milestone 2.4 Ejecting cylinder : FB_Actuator_T3 (proper)

The testing ejector has only a **retracted** limit switch, so its extended end is inferred from a stroke timer.

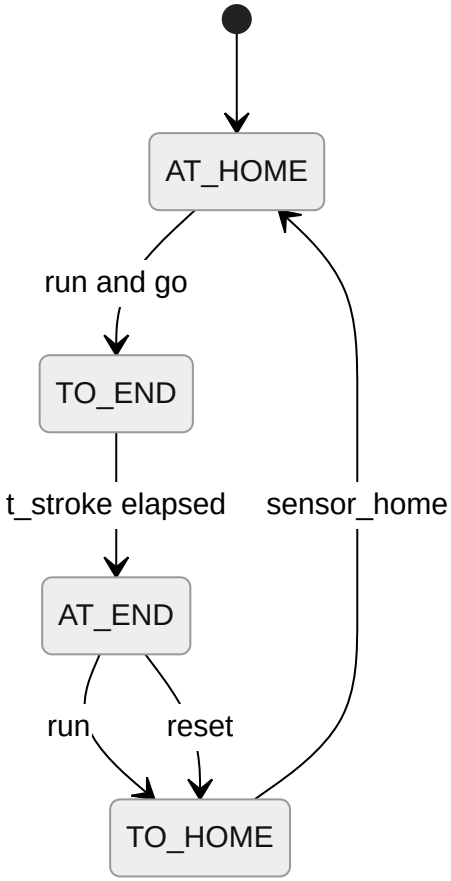

Figure 9. FB_Actuator_T3 proper (2.4)

```
FUNCTION_BLOCK FB_Actuator_T3   // stroke-completing, single retract sensor
VAR_INPUT
    run_mode : BOOL; pause_mode : BOOL; reset_mode : BOOL; error_mode : BOOL;
    sensor_home : BOOL; go : BOOL;
    t_stroke : TIME := T#1s; t_stall : TIME := T#2s; robust : BOOL := FALSE;
END_VAR
VAR_OUTPUT
    at_home : BOOL; to_end : BOOL; at_end : BOOL; to_home : BOOL;
    valve_extend : BOOL; error : BOOL;
END_VAR
VAR state : INT := 0; strokeT : TON; stallT : TON; END_VAR

CASE state OF
0:  IF error_mode THEN state := 4;
    ELSIF reset_mode THEN IF NOT sensor_home THEN state := 3; END_IF;
    ELSIF run_mode AND go THEN state := 1; END_IF;
1:  IF error_mode THEN state := 4; ELSIF strokeT.Q THEN state := 2; END_IF;
2:  IF error_mode THEN state := 4;
    ELSIF reset_mode THEN state := 3;
    ELSIF run_mode THEN state := 3; END_IF;
3:  IF error_mode THEN state := 4; ELSIF sensor_home THEN state := 0; END_IF;
4:  IF reset_mode THEN IF sensor_home THEN state := 0; ELSE state := 3; END_IF; END_IF;
END_CASE;

valve_extend := (state = 1) OR (state = 2);
IF state = 4 THEN valve_extend := FALSE; END_IF;
strokeT(IN := (state = 1), PT := t_stroke);
stallT (IN := robust AND (state = 3), PT := t_stall);
IF stallT.Q THEN state := 4; valve_extend := FALSE; END_IF;

at_home := (state=0); to_end := (state=1); at_end := (state=2);
to_home := (state=3); error := (state=4);
END_FUNCTION_BLOCK
```

Tuning. Ejector extend measured ~0.5 s; `t_stroke` set to `T#0.7s` (just longer than the real travel), `t_stall` left at `T#2s`.

Result, 2.4 (proper). Reset retracted the ejector home. In Run it extended (timed) and retracted (sensed) continuously; a Stop mid-travel completed the stroke before halting. **PASS, 2/2.** Tutor: _____

Date: _____

Milestone 2.5 Ejecting cylinder : FB_Actuator_T3 (robust)

With one sensor, stall is only detectable on the retract leg; a cylinder held while extended is caught on the following retract attempt.

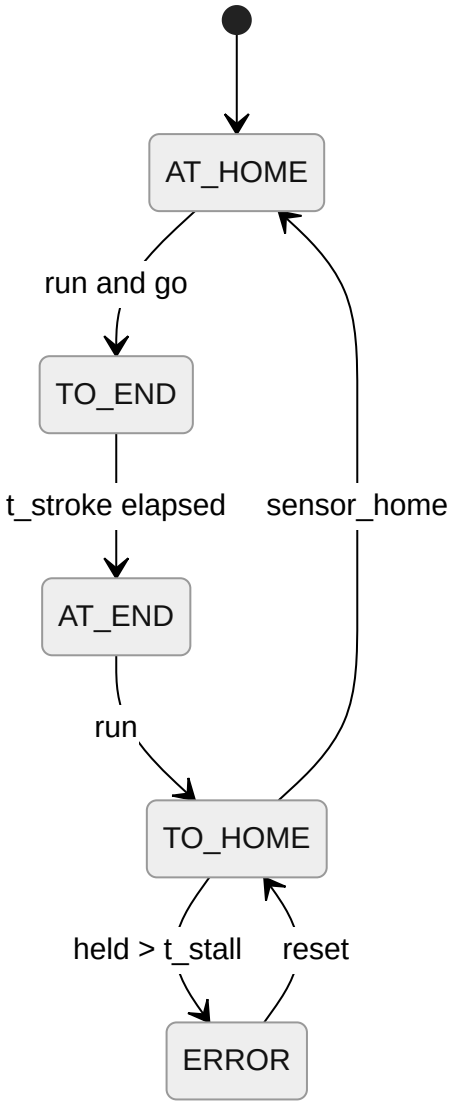

Figure 10. FB_Actuator_T3 robust (2.5)

Result, 2.5 (robust). Holding the ejector against the retract for >2 s raised Q2; Start in error did nothing; Reset then Start resumed. **PASS, 2/2.** Tutor: _____ Date: _____

Milestone 2.6 Full station : Main (proper)

Piece	B2 (not-black)	B5 (tall)	Route
Black	0	–	Bottom slide, no lift
White	1	0 (short)	Bottom slide (after lift + height check)
Red / metal	1	1 (tall)	Top slide, blower ~1 s

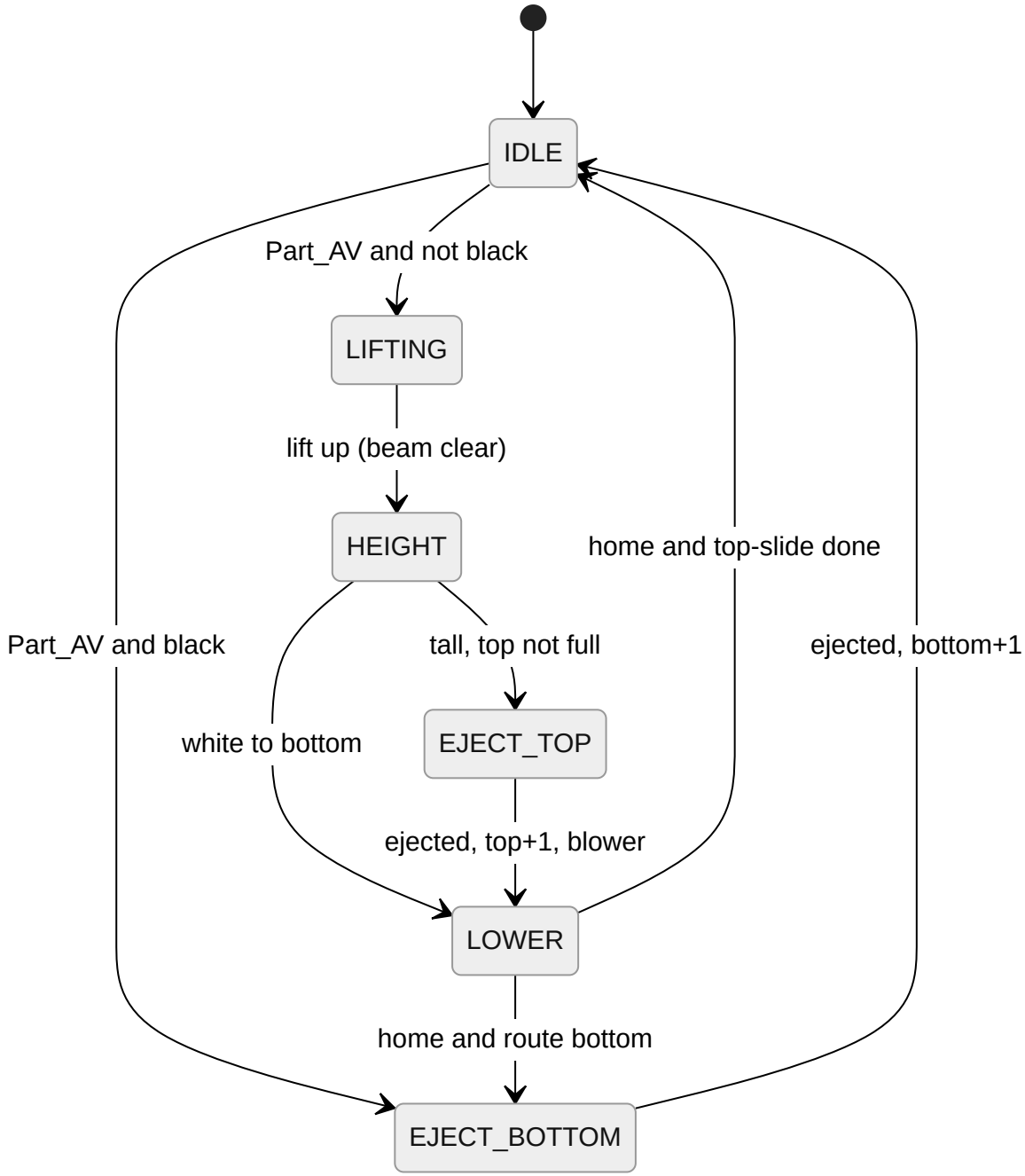

Figure 11. Testing per-piece sequence (2.6)

```
PROGRAM Main_Testing   // 4th block: colour/height sort, blower, slide counters, beam, Q1/Q2
VAR
    Panel : FB_Panel; Lift : FB_Actuator_T1; Ejector : FB_Actuator_T3;
    ps : INT := 0; dest_bottom : BOOL; bottom, top : INT := 0; beam_ctr : INT := 0;
    go_lift, go_ej, blower, black, tall, beam_blocked : BOOL;
    fault, reset_done, beam_fault, proc_fault : BOOL;
    ROBUST : BOOL := TRUE; CAP : INT := 5; BEAM_LIMIT : INT := 3;
END_VAR
reset_done := Lift.at_home AND Ejector.at_home;
Panel(s1_start := Panel_S1, s2_stop := Panel_S2, s4_reset := Panel_S4,
      error_in := fault, reset_done := reset_done);
IF Panel.reset_mode OR Panel.error_mode THEN
    ps := 0; beam_ctr := 0; proc_fault := FALSE; dest_bottom := FALSE; END_IF;
beam_blocked := NOT Station_B4;   // through-beam TRUE=clear; verified on bench
beam_fault := FALSE;
IF beam_blocked AND (ps = 1 OR ps = 5) THEN
    beam_ctr := beam_ctr + 1;
    IF ROBUST AND beam_ctr >= BEAM_LIMIT THEN beam_fault := TRUE; END_IF;
ELSIF NOT beam_blocked THEN beam_ctr := 0; END_IF;
go_lift := FALSE; go_ej := FALSE; blower := FALSE;
IF Panel.run_mode THEN
    CASE ps OF
    0:  IF Part_AV THEN black := NOT Station_B2;
            IF black THEN ps := 4; ELSE ps := 1; END_IF; END_IF;
    1:  IF NOT beam_blocked THEN go_lift := TRUE; IF Lift.at_end THEN ps := 2; END_IF; END_IF;
    2:  tall := Station_B5;
        IF tall THEN IF top < CAP THEN ps := 3; ELSE ps := 5; dest_bottom := FALSE; END_IF;
        ELSE IF ROBUST AND bottom >= CAP THEN proc_fault := TRUE;
             ELSE dest_bottom := TRUE; ps := 5; END_IF; END_IF;
    3:  go_ej := TRUE; blower := TRUE;
        IF Ejector.at_end THEN top := top+1; dest_bottom := FALSE; ps := 5; END_IF;
    4:  go_ej := TRUE; IF Ejector.at_end THEN bottom := bottom+1; ps := 0; END_IF;
    5:  go_lift := TRUE;
        IF Lift.at_home THEN IF dest_bottom THEN ps := 4; ELSE ps := 0; END_IF; END_IF;
    END_CASE;
END_IF;
Lift(run_mode := Panel.run_mode, pause_mode := Panel.pause_mode,
    reset_mode := Panel.reset_mode, error_mode := Panel.error_mode,
    sensor_home := Station_1B2, sensor_end := Station_1B1, go := go_lift,
    t_stall := T#2s, robust := ROBUST);
Ejector(run_mode := Panel.run_mode, pause_mode := Panel.pause_mode,
    reset_mode := Panel.reset_mode, error_mode := Panel.error_mode,
    sensor_home := Station_2B1, go := go_ej,
    t_stroke := T#700ms, t_stall := T#2s, robust := ROBUST);
fault := ROBUST AND (Lift.error OR Ejector.error OR beam_fault OR proc_fault);
Station_1Y1 := Lift.valve_to_home;   Station_1Y2 := Lift.valve_to_end;
Station_2Y1 := Ejector.valve_extend;
Station_3Y1 := blower AND NOT Panel.error_mode;
Station_H1 := Panel.start_light;     Station_H2 := Panel.reset_light;
Station_H3 := (bottom >= CAP);       Station_H4 := Panel.error_mode;
IP_N_FO := Panel.run_mode;
END_PROGRAM
```

Result, 2.6 (proper). Three white pieces lifted to the height sensor then rejected to the bottom slide; three red/metal pieces lifted to the upper position and transferred to the top slide with the blower on ~1 s; three black pieces were rejected straight to the bottom slide without lifting. Reset returned the lift to bottom home and retracted the ejector; Stop mid-lift froze it; Start resumed. **PASS, 2/2.** Tutor: _____ Date: _____

Milestone 2.7 Full station : Main (robust)

Same program with `ROBUST := TRUE`: the slide-full check, the safety-beam counter (`BEAM_LIMIT` scans) and both actuators' stall timers are armed.

> **Result, 2.7 (robust).** Filling the bottom slide (3 black + 2 white) lit Q1. A further white piece with the slide full stopped at entry after the height check and raised Q2 (reset required); a non-white piece with the bottom full was still sent to the top slide and the station stayed in Run. Holding the lift >2 s (up and down) raised Q2. Blocking the safety beam held the lift stationary; blocking it >3 s raised Q2 and entered the error state. **PASS, 3/3.** Tutor: _____ Date: _____

Milestone 2.8 Gantt revision (after Station 2)

Reusing `FB_Actuator_T1` for the lift saved the expected time. Only the call-site I/O (home = down) and the beam polarity needed setting. The single-sensor `FB_Actuator_T3` ejector behaved as designed. S2 SCADA fit within Session 3. On plan.

SCADA : Testing Station · Milestones S2.1, S2.2

Milestone S2.1 SCADA purpose

A supervisory window onto the Testing station in the same style as S1, with the addition of a **colour/height readout** for the piece in process and **top/bottom slide counts**.

Milestone S2.2 SCADA build

Same Designer workflow as S1. The colour label derives from the sensors (`black := NOT B2` ; white = not-black + short; red/metal = not-black + tall); the slide counts bind to the program's `top` / `bottom` variables; a blower icon pulses on `Station_3Y1` .

> **Result, S2.1 / S2.2.** The HMI showed each piece's colour and height as it was tested, animated the lift and ejector, showed pieces arriving on the correct slide with the counts incrementing, and lit Q1 when the bottom slide filled. All required elements present. **PASS, S2.1 1/1, S2.2 4/4.** Tutor: _____ Date: _____

Sorting Station · Milestones 3.1–3.4

Station preamble: I/O table

Name	Addr	Function	Used by
Part_AV	0.00	Workpiece available	Main
Station_B2	0.01	Metallic workpiece	Main (metal)
Station_B3	0.02	Workpiece not black	Main (black := NOT B3)
Station_B4	0.03	Slide full	Main (Q1 + error)
Station_1B1 / 1B2	0.04 / 0.05	Branch 1 retracted / extended	T2 (b1)
Station_2B1 / 2B2	0.06 / 0.07	Branch 2 retracted / extended	T2 (b2)
Station_K1	1.00	Conveyor motor on	Main
Station_1Y1 / 2Y1	1.01 / 1.02	Branch 1 / 2 extend	T2 valves
Station_3Y1	1.03	Stopper retract (release)	T3 valve_extend
Station_H1/H2/H3/H4	1.08–1.11	Start / Reset / Q1 / Q2	Panel + Main

Milestone 3.1 Station purpose

The Sorting station is the third and final station: it **sorts finished workpieces onto three slides by material and colour**. Pieces travel on a conveyor past an inductive **metallic** sensor and an optical **not-black** sensor; each piece is classed as metal, red (non-metal, not black) or black. Two pneumatic **deflector branches** push metal and red pieces onto slides 1 and 2; black pieces run to the end of the belt into slide 3. A retractable **stopper** releases pieces one at a time and a **slide-full** sensor halts the station. The branches reuse `FB_Actuator_T2` and the stopper reuses `FB_Actuator_T3`.

Milestone 3.2 Full station : five blocks (proper)

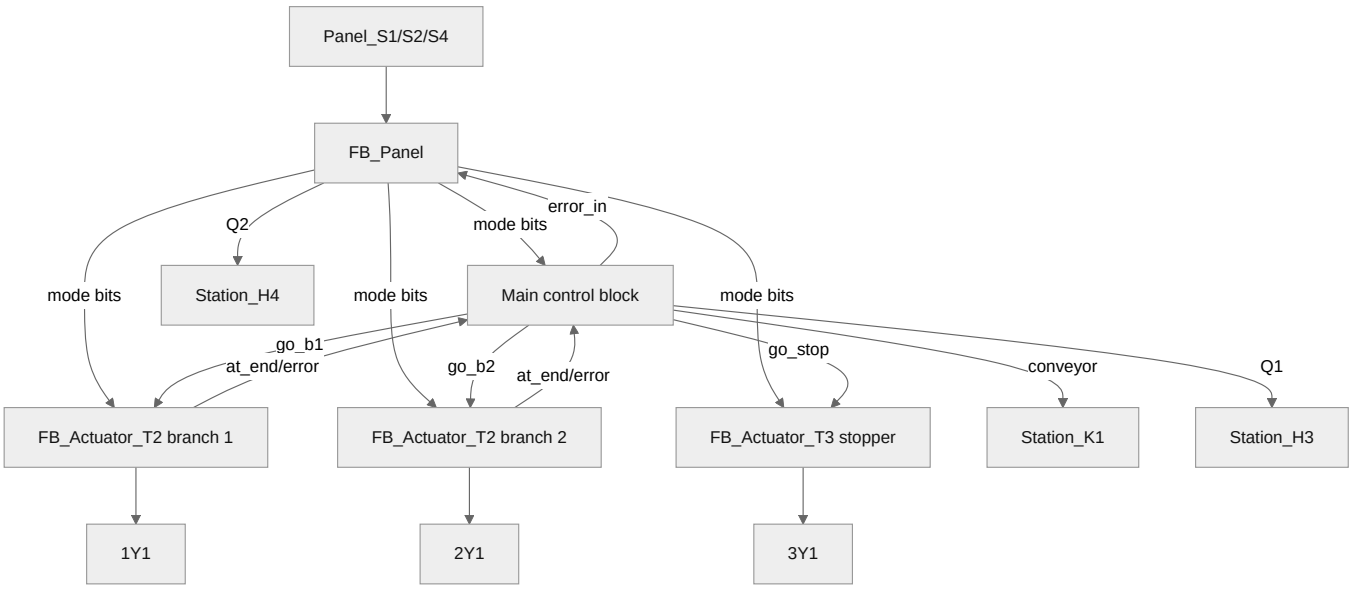

Figure 12. Sorting block diagram (5 blocks, 3.2)

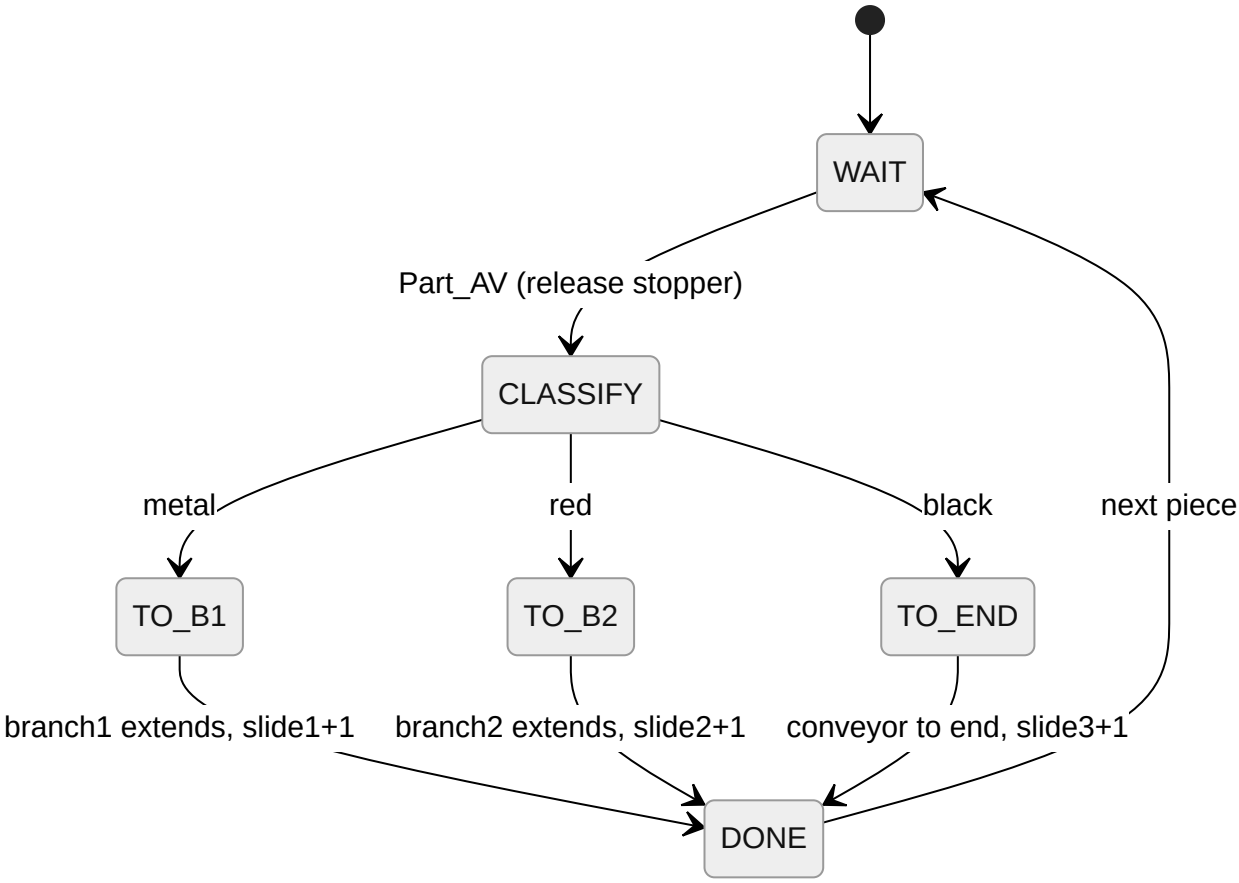

Figure 13. Sorting per-piece sequence (3.2)

```
PROGRAM Main_Sorting   // 5th block: conveyor, sort decision, counters, slide-full, Q1/Q2
VAR
    Panel : FB_Panel; Branch1 : FB_Actuator_T2; Branch2 : FB_Actuator_T2;
    Stopper : FB_Actuator_T3;
    pos : INT := 0; dest : INT := 0; have : BOOL;
    s1c, s2c, s3c : INT := 0;
    conveyor, go_b1, go_b2, go_stop, metal, red, black, slide_full : BOOL;
    fault, reset_done : BOOL;
    ROBUST : BOOL := TRUE; CAP : INT := 5;
END_VAR
reset_done := Branch1.at_home AND Branch2.at_home;
Panel(s1_start := Panel_S1, s2_stop := Panel_S2, s4_reset := Panel_S4,
      error_in := fault, reset_done := reset_done);
IF Panel.reset_mode OR Panel.error_mode THEN have := FALSE; pos := 0; END_IF;
slide_full := Station_B4 OR (s1c >= CAP) OR (s2c >= CAP) OR (s3c >= CAP);
conveyor := FALSE; go_b1 := FALSE; go_b2 := FALSE; go_stop := FALSE;
IF Panel.run_mode AND NOT slide_full THEN
    IF NOT have THEN
        IF Part_AV THEN
            metal := Station_B2; black := NOT Station_B3;
            red := Station_B3 AND NOT Station_B2;
            IF metal THEN dest := 0; ELSIF red THEN dest := 1; ELSE dest := 2; END_IF;
            have := TRUE; pos := 0; go_stop := TRUE;
        END_IF;
    ELSE
        CASE dest OF
        0:  IF pos < 1 THEN conveyor := TRUE; pos := pos+1;
            ELSE go_b1 := TRUE; IF Branch1.at_end THEN s1c := s1c+1; have := FALSE; END_IF; END_I
        1:  IF pos < 2 THEN conveyor := TRUE; pos := pos+1;
            ELSE go_b2 := TRUE; IF Branch2.at_end THEN s2c := s2c+1; have := FALSE; END_IF; END_I
        2:  IF pos < 3 THEN conveyor := TRUE; pos := pos+1;
            ELSE s3c := s3c+1; have := FALSE; END_IF;
        END_CASE;
    END_IF;
END_IF;
Branch1(run_mode := Panel.run_mode, pause_mode := Panel.pause_mode,
    reset_mode := Panel.reset_mode, error_mode := Panel.error_mode,
    sensor_home := Station_1B1, sensor_end := Station_1B2, go := go_b1,
    t_stall := T#2s, robust := ROBUST);
Branch2(run_mode := Panel.run_mode, pause_mode := Panel.pause_mode,
    reset_mode := Panel.reset_mode, error_mode := Panel.error_mode,
    sensor_home := Station_2B1, sensor_end := Station_2B2, go := go_b2,
    t_stall := T#2s, robust := ROBUST);
Stopper(run_mode := Panel.run_mode, pause_mode := Panel.pause_mode,
    reset_mode := Panel.reset_mode, error_mode := Panel.error_mode,
    sensor_home := TRUE, go := go_stop, t_stroke := T#700ms, t_stall := T#2s, robust := ROBUST);
fault := (ROBUST AND (Branch1.error OR Branch2.error))
    OR (Panel.run_mode AND slide_full)
    OR (ROBUST AND Panel.run_mode AND have AND pos >= 1 AND NOT Part_AV);
Station_K1  := conveyor AND NOT Panel.error_mode;
Station_1Y1 := Branch1.valve_extend;   Station_2Y1 := Branch2.valve_extend;
Station_3Y1 := Stopper.valve_extend;
Station_H1  := Panel.start_light;      Station_H2 := Panel.reset_light;
Station_H3  := slide_full;             Station_H4 := Panel.error_mode;
IP_N_FO     := Panel.run_mode;
END_PROGRAM
```

Result, 3.2 (proper). Two red, two metal and two black pieces sorted to slides 2, 1 and 3 respectively. Filling a slide lit Q1 and sent the station to the error state. Reset returned all actuators home; a Stop with a branch mid-travel completed the stroke then halted (home→end and end→home); a Stop with a piece on the conveyor stopped the belt and Start resumed with the piece correctly sorted. **PASS, 3.1 1/1, 3.2 2/2.** Tutor: _____ Date: _____

Milestone 3.3 Full station : five blocks (robust)

Same program with `ROBUST := TRUE` : both branch stall timers are armed and a piece that disappears between the stopper and its slide raises a fault.

Result, 3.3 (robust). Six pieces (2 black, 2 red, 2 metal) placed back-to-back on the conveyor were all sorted correctly. Holding a branch raised Q2 and sent the station to the error state; taking a piece off the conveyor before it reached a slide raised Q2. **PASS, 3/3.** Tutor: _____ Date: _____

Milestone 3.4 Gantt revision (after Station 3)

The two-branch `FB_Actuator_T2` reuse and the stopper `FB_Actuator_T3` dropped in without code changes. On track for L.1–L.3 in Sessions 4–5; no variance from plan.

SCADA : Sorting Station · Milestones S3.1, S3.2

Milestone S3.1 SCADA purpose

A supervisory window onto the Sorting station showing the conveyor, both branches, a **colour/material readout** per piece (metal / red / black from `B2` / `B3`), and a **count on each of the three slides**.

Milestone S3.2 SCADA build

The slide counts bind to the program's counters, with animated transitions as each piece is deflected onto its slide.

> **Result, S3.1 / S3.2.** Material readout updated as each piece was recognised, the correct branch animated, and the matching slide count incremented; a Stop with a piece on the belt stopped the conveyor and Start resumed correct sorting; a full slide lit Q1. **PASS, S3.1 1/1, S3.2 4/4.** Tutor: _____ Date: _____

Integrated Production Line · Milestones L.1–L.3

Milestone L.1 Function-block reuse

The whole line is built from four function blocks plus a per-station Main. L.1 credits each block being the same block on more than one station; the block bodies never change; only the call-site I/O differs.

Function block	Distributing	Testing	Sorting	Type
FB_Panel	✓	✓	✓	control panel
FB_Actuator_T1	arm	lift	–	holdable double-solenoid
FB_Actuator_T2	ejector	–	branch 1 + 2	stroke-completing, 2 sensors
FB_Actuator_T3	–	ejector	stopper	stroke-completing, 1 sensor

`FB_Panel` is byte-for-byte identical on all three stations. The design choice that makes the reuse hold is keeping the vacuum and ejector-pulse valves out of `FB_Actuator_T1` (milestone 1.9), so the arm block drops onto the Testing lift with no baggage.

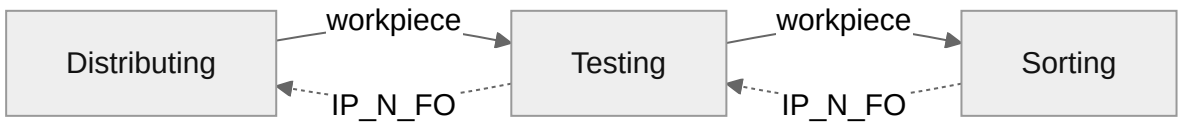

Figure 14. Inter-station handshake (IP_FI / IP_N_FO)

Milestone L.2 Line proper

Only the Sorting station's buttons switch the whole line to run/reset/pause; the Sorting panel's mode is distributed to the other two stations, which ignore their local buttons. Adjacent suitcases are wired station-to-station with `IP_N_FO` (occupied) read as the upstream station's `IP_FI` (succeeding station free), so a workpiece is handed over only when the next station is ready.

> **Result, L.2.** The Sorting Start/Stop/Reset controlled all three stations. With the swivel arm and lift set in-between, the master Reset homed every station. Two red, two metal, two black and two white pieces were processed through the line; pausing and resuming three times (arm in-between, lift in-between, piece on the conveyor) resumed each time. **PASS, 5/5.** Tutor: _____ Date: _____

Milestone L.3 Line robust

Error (Q2) propagates **upstream only**: a fault at a station stops everything feeding it. This is implemented by ORing each downstream station's `error_mode` into the upstream stations' `FB_Panel.error_in`.

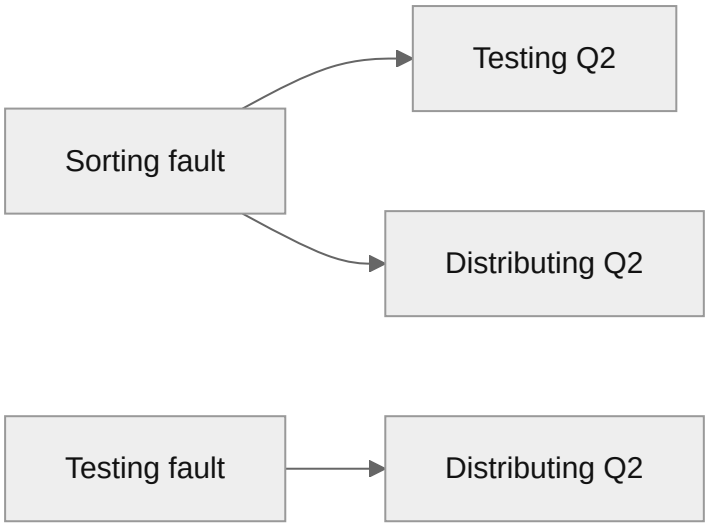

Figure 15. Upstream Q2 propagation (L.3)

Faulting station	Stations forced to Q2
Sorting	Sorting + Testing + Distributing (all 3)
Testing	Testing + Distributing (2)
Distributing	Distributing only (1)

Result, L.3. A piece missing from the Sorting station lit all three Q2 lights. The safety beam blocked >3 s lit two Q2 lights (Testing + Distributing). The swivel arm held lit one Q2 light (Distributing). Every error was cleared using the Sorting station buttons only. Each station also ran robustly per its own specification. **PASS, 6/6.** Tutor: _____ Date: _____

Appendix: Design Verification

Before each lab, every station's control logic was checked step by step against its finite-state diagram and the station test protocol. Each numbered protocol step was walked through the diagram to confirm the state, valve outputs and lights matched the expected result. The ladder/ST typed into the PLC is a direct transcription of these checked state machines.

Protocol group checked	Covers
1.4 panel FSM	power-on, all transitions, invalid transitions, one-light interlock
1.5 / 1.6 swivel arm	pause/reset/resume; bidirectional stall → Q2
1.7 / 1.8 ejection cylinder	stroke completion on pause; bidirectional stall → Q2
1.9 / 1.10 distributing	5-piece cycle; mid-delivery pause; hold / missing-piece → Q2
2.2–2.5 lift & ejector	FB_Actuator_T1 reuse; single-sensor T3 strokes
2.6 / 2.7 testing	white/black→bottom, red-metal→top+blower; slide-full Q1; white-full Q2; hold; beam
3.1 / 3.2 sorting	metal/red/black→3 slides; full→Q1+error; back-to-back; branch-held / piece-removed Q2
L.2 / L.3 line	master control; upstream Q2 propagation (3 / 2 / 1 lights)

> **Verification result.** Every FSM matched its test protocol step by step across all groups above. The design was fixed only once each protocol sequence checked out, so lab time went to demonstration rather than debugging.

Deployment

Code was deployed to each PLC either by importing one IEC 61131-10 (PLCopen) XML per station (Sysmac ≥ v1.30, File > Import) or, as a fallback, by pasting the plain Structured Text POUs and the I/O-table variable names into the Sysmac I/O map. Stall/stroke timers are FB inputs and were tuned at the bench; the Testing safety-beam polarity (`beam_blocked := NOT Station_B4`) was confirmed on the real station.

Marks Ledger

#	Milestone	Session	Marks	Result
P.1	Gantt chart + strategy	1	1	✓
1.1	Tick-tock sampler	1	1	✓
1.2	Station purpose	1	1	✓
1.3	Interface description	1	1	✓
1.4	Control panel proper	1	2	✓
1.5	Swivel arm proper	1	2	✓
1.6	Swivel arm robust	1	2	✓
1.7	Ejection cylinder proper	1	2	✓
1.8	Ejection cylinder robust	1	2	✓
1.9	Station proper	2	3	✓
1.10	Station robust	2	3	✓
1.11	Revise Gantt	2	1	✓
S1.1	SCADA purpose	2	1	✓
S1.2	SCADA build	2	4	✓
2.1	Station purpose	2	1	✓
2.2	Lift proper	2	1	✓
2.3	Lift robust	2	1	✓
2.4	Ejector proper	3	2	✓
2.5	Ejector robust	3	2	✓
2.6	Station proper	3	2	✓
2.7	Station robust	3	3	✓
2.8	Revise Gantt	3	1	✓
S2.1	SCADA purpose	3	1	✓
S2.2	SCADA build	3	4	✓
3.1	Station purpose	4	1	✓
3.2	Station proper	4	2	✓
3.3	Station robust	4	3	✓
3.4	Revise Gantt	4	1	✓
S3.1	SCADA purpose	4	1	✓
S3.2	SCADA build	4	4	✓
L.1	FB reuse across 3 stations	4	3	✓
L.2	Line proper	5	5	✓

L.3	Line robust	5	6	✓
Total			**70**	✓

Every milestone demonstrated in the laboratory and signed off by the tutor in the session shown. Prerequisite order respected throughout; no session exceeded the 15-mark cap.